



CHƯƠNG 6

Quản trị cơ sở dữ liệu

Triển khai và quản trị hệ thống phần mềm



Giới thiệu nội dung

Mục tiêu chương

- Giải thích vòng đời quản lý tài khoản CSDL, nguyên tắc quyền hạn tối thiểu (PoLP) và mô hình RBAC.
- Phân biệt cơ chế xác thực, phân quyền của MySQL và PostgreSQL.
- Triển khai MySQL/PostgreSQL bằng Docker Compose; thực hành tạo tài khoản, phân quyền và mã hóa dữ liệu.

Nội dung chính

- 1 Quản lý tài khoản người dùng CSDL**
- 2 Phân quyền và mô hình RBAC**
- 3 MySQL và PostgreSQL: cơ chế đặc thù**
- 4 Thực hành quản trị CSDL với Docker Compose**

Vì sao cần quản lý tài khoản CSDL?

- Quản lý tài khoản người dùng (User Account Management – UAM) không chỉ là cấp tên đăng nhập và mật khẩu.
- Là quy trình kỹ thuật và quản lý nhằm xác định, thiết lập và duy trì các thực thể có quyền tương tác với dữ liệu.
- Được xem là **“chốt chặn” đầu tiên** và quan trọng nhất để bảo vệ tài sản số của tổ chức.

Câu hỏi mở đầu

Nếu một công ty để tất cả lập trình viên, nhân viên và ứng dụng web dùng chung một tài khoản **root**, điều gì có thể xảy ra khi có sự cố?

Bản chất của một tài khoản CSDL

- Là một định danh duy nhất (Unique Identity) được DBMS công nhận – khác tài khoản hệ điều hành, chỉ tồn tại trong phạm vi công cụ quản trị dữ liệu.
- Lưu trữ ba loại thông tin cơ bản: **đanh tính** (username), **phương thức xác thực** (password/plugin), **trạng thái** (active/locked).
- Bản chất: thiết lập một “hợp đồng tin cậy” giữa người dùng và hệ thống – mọi truy xuất dữ liệu đều phải qua kiểm tra danh tính nghiêm ngặt.

Vai trò và ý nghĩa của quản lý tài khoản

- **Xác lập ranh giới bảo mật:** tách biệt cá nhân và ứng dụng khi cùng truy cập một CSDL dùng chung.
- **Kiểm soát tài nguyên:** giới hạn số kết nối, thời gian, bộ nhớ – tránh một tài khoản làm sập cả hệ thống.
- **Tạo cơ sở cho phân quyền:** không có tài khoản riêng biệt thì không thể áp dụng nguyên tắc “quyền hạn tối thiểu”.
- **Đảm bảo tính không thể chối bỏ (non-repudiation):** xác định chính xác ai đã làm gì, tại thời điểm nào.

Ý nghĩa với doanh nghiệp

Hơn 60% vụ rò rỉ dữ liệu bắt nguồn từ bên trong tổ chức; quản lý tài khoản lỏng lẻo còn vi phạm các yêu cầu pháp lý như GDPR, ISO 27001 hay Luật An ninh mạng.

Vòng đời quản lý tài khoản (life-cycle management)

- 1 **Khởi tạo (onboarding):** xác định vai trò công việc, đặt username theo đúng quy chuẩn.
- 2 **Cấp quyền:** gán các quyền cần thiết dựa trên mô hình RBAC.
- 3 **Giám sát:** theo dõi lịch sử đăng nhập, các lệnh SQL bất thường.
- 4 **Duy trì/cập nhật:** thay đổi mật khẩu định kỳ, cập nhật quyền khi đổi vị trí công việc.
- 5 **Thu hồi (offboarding):** khóa/xóa tài khoản ngay khi nhân viên nghỉ việc hoặc dự án kết thúc.

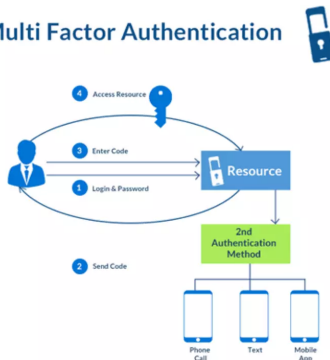
Lưu ý

Giai đoạn thu hồi thường bị bỏ quên nhất, tạo ra các “tài khoản ma” cực kỳ nguy hiểm.

Các mô hình xác thực hiện đại

- **Password-based:** phổ biến nhất nhưng dễ bị tấn công brute-force.
- **Certificate-based:** dùng khóa SSL/TLS để xác thực máy chủ và người dùng.
- **Tập trung (LDAP/Active Directory):** dùng chung tài khoản công ty, DBA quản lý tập trung một nơi.
- **Đa yếu tố (MFA):** khó triển khai trực tiếp trên SQL, thường áp dụng qua proxy hoặc cổng trung gian.

Multi Factor Authentication



Ba loại tài khoản phổ biến trên DBMS

- **Administrative (Root/postgres/SA):** tài khoản “quyền năng nhất”, có thể xóa toàn bộ CSDL – chỉ dành cho DBA cấp cao.
- **End-user:** tài khoản cho nhân viên, ví dụ `ke_toan_01`, `sale_manager`.
- **Service/Application:** tài khoản cho ứng dụng, ví dụ `web_app_user` – thường không đăng nhập từ console, chỉ kết nối từ một IP server cố định.

Tình huống: hậu quả khi dùng chung tài khoản root

- Một công ty thương mại điện tử có một CSDL MySQL dùng chung.
- Tất cả mọi người (lập trình viên, kế toán, ứng dụng web) đều dùng chung tài khoản `root` với mật khẩu `123456`.
- Một thực tập sinh chạy nhầm lệnh `DROP DATABASE`.

Hậu quả

Không ai biết ai đã làm, dữ liệu mất trắng, ứng dụng sập – vì **không có tài khoản riêng biệt để truy vết trách nhiệm**.

Cách quản lý tài khoản đúng

Ví dụ

- Đặt tên theo quy tắc `[vai trò]_[tên]`: ví dụ `dev_tuan`, `admin_hung`, `audit_ketoan`.
- Cấp đúng quyền cần thiết: `app_web` chỉ `SELECT`, `INSERT`, `UPDATE` trên bảng sản phẩm.
- Ràng buộc host kết nối, ví dụ `dev_tuan@'192.168.1.%'` thay vì cho phép mọi nơi (%).
- Áp dụng chính sách mật khẩu mạnh: tối thiểu 12 ký tự, đủ chữ hoa, chữ thường, số và ký tự đặc biệt.

Từ xác thực đến phân quyền: nguyên tắc PoLP

- Sau khi xác thực thành công, DBMS thực hiện bước ủy quyền (Authorization): xác định các đặc quyền (privileges) trên tài nguyên cụ thể.
- Nếu quản lý tài khoản là “chốt chặn” đầu tiên, phân quyền chính là “màng lọc” tinh vi bảo vệ tính toàn vẹn và bảo mật dữ liệu.

Nguyên tắc quyền hạn tối thiểu (PoLP)

Mỗi người dùng, chương trình hoặc tiến trình chỉ được cấp những quyền hạn vừa đủ để hoàn thành nhiệm vụ được giao – không hơn không kém.

- Nếu tài khoản ứng dụng chỉ cần đọc dữ liệu mà lại có quyền **DELETE**, một lỗ hổng **SQL Injection** nhỏ cũng có thể xóa sạch toàn bộ dữ liệu.

Hai nhóm quyền: hệ thống và đối tượng

Loại quyền	Mô tả và ví dụ
Quyền hệ thống	Tác động trên toàn bộ instance CSDL; thường chỉ dành cho DBA. Ví dụ: <code>CREATE USER</code> , <code>ALTER SYSTEM</code> , <code>CREATE ANY TABLE</code> .
Quyền đối tượng	Thực thi trên đối tượng cụ thể (bảng, view, hàm). Ví dụ: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>EXECUTE</code> .

RBAC: kiểm soát truy cập dựa trên vai trò

- Khi hệ thống có hàng trăm người dùng, phân quyền cho từng cá nhân là một “cơn ác mộng quản trị”.
- RBAC (Role-Based Access Control) chèn một lớp trung gian gọi là **vai trò (role)**: quyền được gán cho vai trò → vai trò được gán cho người dùng.
- Ưu điểm: khi nhân viên đổi bộ phận, DBA chỉ cần thu hồi role cũ và gán role mới, không cần rà soát từng lệnh `GRANT/REVOKE` riêng lẻ.

Ví dụ

Tạo vai trò kế toán, cấp quyền `SELECT` trên bảng `Luong`. Bất kỳ nhân viên nào thuộc phòng kế toán sẽ được gán vai trò này.

Phân quyền mức hạt mịn: cột và dòng

- Đôi khi quyền hạn ở mức bảng vẫn quá rộng – cần kiểm soát sâu hơn.
- **Quyền mức cột (column-level security):** nhân sự xem bảng `NhanVien` nhưng ẩn cột `SoDienThoai` hoặc `MatKhau`.
- **Quyền mức dòng (row-level security – RLS):** trưởng phòng kinh doanh miền Bắc chỉ thấy các dòng dữ liệu của khách hàng thuộc khu vực miền Bắc, dù tất cả nằm chung bảng `KhachHang`.

WITH GRANT OPTION: cơ chế lan truyền quyền

- Khi cấp quyền, có tùy chọn `WITH GRANT OPTION`: cho phép người nhận quyền tiếp tục cấp lại quyền đó cho một người thứ ba.

Lưu ý

Đây là “con dao hai lưỡi”. Nếu không kiểm soát chặt chẽ, quyền hạn sẽ **lan truyền ngoài tầm kiểm soát** của DBA ban đầu, tạo ra các lỗ hổng bảo mật dây chuyền.

Case study: rò rỉ dữ liệu do vi phạm PoLP

Sự cố năm 2021

- Một ngân hàng lớn tại Mỹ: nhân viên IT hạ tầng (chỉ cần quyền truy cập server) lại sở hữu tài khoản CSDL có quyền `SELECT` trên toàn bộ bảng thông tin thẻ tín dụng.
- Máy tính của nhân viên bị nhiễm mã độc → kẻ tấn công trích xuất dữ liệu của **hơn 10 triệu khách hàng**.
- **Sai lầm:** vi phạm PoLP – cấp quyền không liên quan đến vai trò nghiệp vụ.
- **Giải pháp:** áp dụng RBAC nghiêm ngặt, định kỳ thực hiện “audit quyền hạn” để thu hồi các quyền không còn cần thiết.

MySQL: định danh user@host độc đáo

- Trong MySQL, một tài khoản không chỉ là cái tên – là sự kết hợp giữa **tên người dùng** và **địa chỉ máy khách (host)**.
- Cùng người dùng `admin`: kết nối từ `localhost` có toàn quyền, kết nối từ một IP lạ chỉ có quyền đọc.
- Đây là một lớp bảo mật bổ sung cực kỳ hữu ích để **giới hạn phạm vi tấn công**.

MySQL: xác thực, roles và bảng hệ thống

- Từ bản 8.0, plugin xác thực mặc định là `caching_sha2_password` (an toàn hơn `mysql_native_password` cũ); chính sách mật khẩu cấu hình qua component `validate_password`.
- Roles (từ bản 8.0) là các tài khoản “không có quyền đăng nhập” dùng để tập hợp quyền; phải chạy `SET DEFAULT ROLE` sau khi gán thì quyền mới có hiệu lực ở lần đăng nhập sau.
- Toàn bộ tài khoản và quyền được lưu trong database `mysql`: các bảng `user`, `db`, `tables_priv`, `columns_priv`.

MySQL: cú pháp tạo tài khoản và cấp/thu hồi quyền

```
-- Tạo người dùng với ràng buộc IP
CREATE USER 'dev_tuan'@'192.168.1.50' IDENTIFIED BY 'StrongPassword123!';
-- Cấp quyền đối tượng
GRANT SELECT, INSERT ON sales_db.* TO 'dev_tuan'@'192.168.1.50';
-- Thu hồi quyền
REVOKE INSERT ON sales_db.* FROM 'dev_tuan'@'192.168.1.50';
```

PostgreSQL: triết lý “role” đồng nhất

- Nếu MySQL coi trọng vị trí kết nối, PostgreSQL coi trọng **tính logic, phân cấp và sự đồng nhất**.
- Mọi thực thể (người dùng, nhóm) đều được gọi chung là **role**.
- Role có thuộc tính **LOGIN** → đóng vai trò người dùng; role chứa các role khác → đóng vai trò nhóm.
- Cách tiếp cận này giúp thiết lập mô hình RBAC trở nên cực kỳ tự nhiên và mạnh mẽ.

PostgreSQL: ba cấp quyền để truy cập một bảng

- 1 Quyền kết nối vào database (`CONNECT`).
- 2 Quyền sử dụng schema chứa bảng đó (`USAGE`).
- 3 Quyền thao tác trên bảng cụ thể (`SELECT`, `UPDATE`...).

Lưu ý

Chỉ `GRANT SELECT ON TABLE` mà quên `USAGE ON SCHEMA` → người dùng nhận lỗi **“Permission denied”** ngay lập tức.

PostgreSQL: pg_hba.conf – xác thực theo vị trí kết nối

- Khác MySQL (quản lý host ngay trong câu lệnh SQL), Postgres dùng một file cấu hình hệ thống gọi là `pg_hba.conf` (Host-Based Authentication).
- File này định nghĩa: ai, từ đâu, vào database nào, dùng phương thức xác thực gì.
- Phương thức: `scram-sha-256` (mạnh, khuyến nghị), `md5` (cũ, kém an toàn), `trust` (rất nguy hiểm – không cần mật khẩu).
- Là lớp bảo vệ vòng ngoài, được kiểm tra trước khi lệnh SQL được xử lý.

PostgreSQL: quyền mặc định cho đối tượng tương lai

- Lệnh `ALTER DEFAULT PRIVILEGES` giải quyết bài toán quản trị quyền cho các **đối tượng chưa tồn tại**.
- Ví dụ: “mọi bảng do admin tạo ra trong tương lai, role `readonly_group` sẽ tự động có quyền `SELECT`”.
- Đây là điểm ưu việt so với việc phải cấp lại quyền thủ công mỗi khi có bảng mới.

PostgreSQL: cú pháp tạo role và cấp quyền

```
-- Tạo role nhóm
CREATE ROLE readonly_role;
-- Cấp quyền schema và table
GRANT USAGE ON SCHEMA public TO readonly_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;
-- Gán người dùng vào nhóm
GRANT readonly_role TO user_a;
```

So sánh nhanh: MySQL và PostgreSQL

Tiêu chí	MySQL	PostgreSQL
Định danh tài khoản	<code>user@host</code>	role (thuộc tính <code>LOGIN</code>)
Cấu hình xác thực	Trong câu lệnh SQL	File <code>pg_hba.conf</code>
Lưu trữ quyền hạn	Bảng hệ thống <code>mysql.*</code>	Phân cấp role + <code>ALTER DEFAULT PRIVILEGES</code>
Roles	Từ bản 8.0, cần <code>SET DEFAULT ROLE</code>	Role vừa là user vừa là group (đồng nhất)

Kiến trúc thực hành: bốn dịch vụ trên Docker Compose

- Triển khai MySQL và PostgreSQL cùng công cụ quản trị web bằng một file `docker-compose.yml`.
- `mysql` (`mysql:8.0`, cổng `3306`) và `postgres` (`postgres:16`, cổng `5432`) – hai DBMS chính.
- `phpmyadmin` (cổng `8080`) quản trị MySQL; `pgadmin` (cổng `8081`) quản trị PostgreSQL.
- Dữ liệu lưu trong volume riêng (`mysql_data`, `postgres_data`) để không mất khi container khởi động lại.

Trước khi dùng thực tế

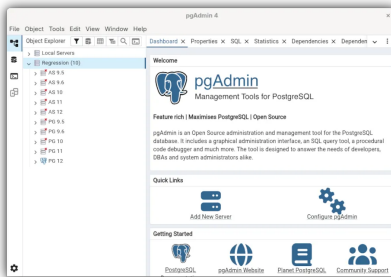
Đổi mật khẩu mặc định trong file compose; chỉ mở cổng `8080/8081` khi cần và giới hạn qua firewall hoặc reverse proxy.

Khai báo dịch vụ MySQL trong docker-compose.yml

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql_db
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: myapp
      MYSQL_USER: myuser
      MYSQL_PASSWORD: mypassword
    ports:
      - "3306:3306"
```

Truy cập công cụ quản trị: phpMyAdmin và pgAdmin

- Khởi tạo container: `docker compose up -d`; kiểm tra dịch vụ và cổng đã hoạt động.
- phpMyAdmin (MySQL): <http://localhost:8080> – đăng nhập `myuser/mypassword`.
- pgAdmin (PostgreSQL): <http://localhost:8081> – đăng nhập `admin@example.com/adminpassword`, tạo kết nối tới host `postgres`, cổng `5432`.



Bài thực hành trên phpMyAdmin (MySQL): yêu cầu chính

- Quản lý người dùng và phân quyền theo nguyên tắc Least Privilege; thiết lập chính sách mật khẩu mạnh.
- Bảo vệ chống SQL Injection; mã hóa dữ liệu nhạy cảm (cột và kết nối).
- Kích hoạt logging/auditing để theo dõi hoạt động; nhận diện và khắc phục cấu hình bảo mật yếu.

Ví dụ

Tạo `university_db` với bảng `users` (có cột nhạy cảm `ssn`), `students`, `grades`; tạo `app_user` chỉ được `SELECT/INSERT/UPDATE`, `report_user` chỉ được `SELECT`.

Bài thực hành trên pgAdmin (PostgreSQL): điểm khác biệt

- Cùng mục tiêu Least Privilege, nhưng thao tác qua **Grant Wizard** thay vì tick checkbox như phpMyAdmin.
- Dùng **Row Level Security (RLS)** – điểm mạnh riêng của PostgreSQL, không có trong bài thực hành MySQL.
- Mã hóa dữ liệu nhạy cảm bằng extension `pgcrypto` thay vì hàm `AES_ENCRYPT` như MySQL.
- Chính sách mật khẩu: `ALTER USER app_user PASSWORD EXPIRE;` kết hợp cấu hình `pg_hba.conf` dùng SCRAM-SHA-256.

Bài tập tổng hợp: phân quyền ba tài khoản theo vai trò

- Bối cảnh: CSDL `university_db` cần ba tài khoản – `app_web` (đọc/cập nhật dữ liệu sinh viên, điểm số), `daotao_01` (xem/sửa điểm, không đổi dữ liệu tài khoản), `audit_01` (chỉ đọc để đối soát).
- **Bước 1 – Làm theo mẫu:** tạo ba tài khoản đúng quy tắc `[vai trò]_[định danh]`, ràng buộc host/kết nối, mật khẩu tối thiểu 12 ký tự đủ độ phức tạp.
- **Bước 2 – Thay đổi có kiểm soát:** cấp đúng quyền theo từng nhóm (không thừa `DELETE/DROP`), đăng nhập thử bằng từng tài khoản và ghi lại thông báo lỗi khi cố tình chạy lệnh vượt quyền.

Bài tập tổng hợp: dùng role và xử lý khi đổi vị trí công việc

- **Bước 3 – Kết hợp:** tạo một role gom quyền `SELECT` trên cả ba bảng, gán cho `audit_01` thay vì cấp trực tiếp từng bảng – so sánh số lệnh `GRANT` cần dùng giữa hai cách.
- **Bước 4 – Vận dụng tình huống mới:** `daotao_01` chuyển sang mảng công việc khác không còn liên quan đến điểm số, trong khi công ty tuyển một nhân viên đào tạo mới.

Yêu cầu

Thiết kế quy trình xử lý tài khoản `daotao_01` cũ (không chỉ đơn thuần xóa quyền) và quy trình cấp tài khoản mới, sao cho không để lại “**tài khoản ma**” và vẫn đảm bảo nguyên tắc quyền hạn tối thiểu.

Câu hỏi thảo luận

- Nhân viên IT chuyển sang bộ phận Marketing – vì sao nên thu hồi toàn bộ tài khoản cũ thay vì chỉ xóa quyền?
- Đề xuất ít nhất 3 tiêu chí đánh giá một hệ thống quản lý tài khoản CSDL là “an toàn”.
- Nếu là DBA, bạn thiết kế quy trình “audit quyền hạn” định kỳ như thế nào để phát hiện sớm quyền thừa?

Tổng kết chương

Những điểm cần ghi nhớ

- Tài khoản CSDL có vòng đời gồm khởi tạo, cấp quyền, giám sát và thu hồi; bỏ quên thu hồi tạo ra “tài khoản ma”.
- Nguyên tắc quyền hạn tối thiểu (PoLP) chỉ cấp đúng quyền cần thiết; RBAC gán quyền cho vai trò rồi gán vai trò cho người dùng.
- MySQL định danh tài khoản theo `user@host`; PostgreSQL coi mọi thực thể là role và dùng `pg_hba.conf` để xác thực theo vị trí kết nối.
- MySQL, PostgreSQL cùng phpMyAdmin, pgAdmin triển khai đồng thời bằng Docker Compose để thực hành tạo tài khoản, phân quyền và logging.